

LAB8 – INF1316 – Sistemas Operacionais

Semáforos

1. Identificação

Nome: Caio Faria Brito Martins Chaves

Matrícula: 2410162

Nome: Lucas Hufnagel Gromann de Araujo Goes

Matrícula: 2410845

2. Objetivo

O objetivo deste laboratório é estudar, na prática, os problemas de concorrência sobre dados compartilhados e a sua correção por meio de semáforos, utilizando linguagem C em ambiente Unix/Linux.

O trabalho propõe a implementação de duas versões de um mesmo problema:

- uma versão sem semáforo, para evidenciar o erro de concorrência;
- uma versão com semáforo (biblioteca `semaphore.h`), para corrigir o problema.

Em ambos os casos, foi utilizado um vetor global de 10.000 posições inteiras, inicializado com o valor 10, sobre o qual 100 tarefas concorrentes (threads pthreads) executam alterações: a tarefa i subtrai i de todas as posições do vetor quando i é ímpar e soma i quando i é par.

Além da implementação, o laboratório teve como finalidade:

- verificar se o acesso concorrente sem proteção provoca erros no vetor;
- identificar a região crítica do programa;
- corrigir o problema utilizando um semáforo binário (`sem_wait/sem_post`);
- verificar automaticamente se todas as posições do vetor final possuem o valor esperado;
- comparar o comportamento das duas versões.

Esse objetivo dá continuidade ao LAB7, no qual foi observado que threads compartilham o mesmo espaço de endereçamento e que o acesso concorrente a dados globais exige exclusão mútua quando envolve mais de uma instrução de máquina.

3. Estrutura do Programa

O trabalho foi dividido em dois programas.

3.1 Programa sem semáforo

O primeiro programa (`sem_semaforo.c`) cria as 100 tarefas como threads pthreads, compiladas com a opção `-pthread`.

De forma geral:

- o vetor global é criado e inicializado com 10 pelo processo principal;
- são criadas 100 threads trabalhadoras com `pthread_create()`;

- a tarefa de número i percorre todas as 10.000 posições do vetor e aplica a operação: `vetor[i] = vetor[i] + valor;` /* valor = +id (par) ou -id (impar) */
- nenhuma proteção de exclusão mútua é utilizada;
- o programa usa `pthread_join()` para aguardar o término das threads;
- ao final, o programa verifica automaticamente se todas as posições possuem o valor esperado.

Para garantir que as threads realmente disputem o vetor ao mesmo tempo, todas são criadas bloqueadas em uma variável de condição e liberadas juntas pela `main`. Sem esse cuidado, uma thread poderia terminar todo o seu trabalho antes da próxima ser criada, mascarando o erro de concorrência.

3.2 Programa com semáforo

O segundo programa (`com_semaforo.c`) tem exatamente a mesma estrutura, mas inclui um semáforo binário da biblioteca `semaphore.h`, criado com `sem_init(&semaforo, 0, 1)`.

De forma geral:

- o vetor e as 100 threads são criados da mesma forma que na primeira versão;
- a atualização de cada posição do vetor passa a ser protegida pelo semáforo:


```
sem_wait(&semaforo);          /* Down */
vetor[i] = vetor[i] + valor;
sem_post(&semaforo);          /* Up */
```
- ao final, o semáforo é liberado com `sem_destroy()` e a mesma verificação automática é executada.

Importante: o semáforo protege apenas a atualização de uma posição por vez, e não o laço inteiro. Assim, continua havendo concorrência entre as tarefas (as threads se intercalam livremente entre uma posição e outra), mas a operação ler–modificar–escrever de cada posição passa a ser atômica.

4. Solução

4.1 Valor esperado

Como toda tarefa percorre o vetor inteiro, ao final todas as posições devem ter o mesmo valor:

$$10 + (2 + 4 + \dots + 100) - (1 + 3 + \dots + 99) = 10 + 2550 - 2500 = 60$$

Ou seja, cada uma das 10.000 posições deve valer 60. Qualquer posição com valor diferente de 60 indica que houve problema de concorrência.

4.2 Versão sem semáforo

Na versão sem semáforo, cada thread executa `vetor[i] = vetor[i] + valor` em todas as posições, sem nenhuma proteção.

O ponto principal observado é que essa operação não é atômica. Ela é composta por três passos: ler o valor da memória, somar e escrever de volta. Se duas threads leem a mesma posição antes que uma delas escreva o resultado, a atualização de uma delas é perdida (lost update).

Esse tipo de situação caracteriza uma condição de corrida, exatamente como discutido no LAB7 para a versão com threads sem sincronização.

O erro de concorrência ocorreu já com 100 tarefas, não sendo necessário aumentar para 1000. Saída de uma execução:

```
SEM SEMAFORO (100 tarefas)
Valor esperado em cada posicao: 60
Erro na posicao 33: valor 12 (esperado 60)
Erro na posicao 2980: valor 159 (esperado 60)
Erro na posicao 2981: valor 159 (esperado 60)
Erro na posicao 2982: valor 159 (esperado 60)
Erro na posicao 2983: valor 159 (esperado 60)
Erro na posicao 2984: valor 159 (esperado 60)
Erro na posicao 2985: valor 111 (esperado 60)
Erro na posicao 2992: valor 111 (esperado 60)
Erro na posicao 3146: valor 111 (esperado 60)
...
Total de posicoes com erro: 270 de 10000
Houve erro de concorrencia!
```

Sobre esse resultado, vale destacar:

- as posições com erro e a quantidade de erros variam a cada execução, pois dependem de como o escalonador intercala as threads. Em três execuções seguidas foram obtidas 197, 76 e 763 posições erradas;
- os erros aparecem em “blocos” de posições consecutivas (ex: 2980–2985, 3146–3157), o que mostra o momento em que duas threads percorriam a mesma região do vetor simultaneamente: enquanto estavam sobrepostas, várias atualizações seguidas foram perdidas;
- o valor errado pode ser maior ou menor que 60 (ex: 12, 111, 159), dependendo de qual atualização foi perdida (uma soma ou uma subtração).

4.3 Versão com semáforo

Na versão com semáforo, a região crítica identificada é a atualização de cada posição do vetor. Essa região foi protegida por um semáforo binário (valor inicial 1), que garante exclusão mútua: apenas uma thread por vez executa a sequência ler–modificar–escrever sobre uma posição.

Com isso, o lost update é eliminado, sem impedir a concorrência: entre uma posição e outra as threads continuam se intercalando livremente.

Resultado obtido com 100 tarefas:

```
COM SEMAFORO (100 tarefas)
Valor esperado em cada posicao: 60
Total de posicoes com erro: 0 de 10000
Nao houve erro de concorrencia.
```

Em todas as execuções realizadas (inclusive com 1000 tarefas) o resultado foi 0 posições com erro: todas as 10.000 posições terminaram com o valor esperado.

4.4 Escolha do semáforo

O enunciado também permite usar semáforo nomeado: `sem_t *sem = sem_open("/lab8", O_CREAT, 0666, 1)`, com `sem_close(sem)` e `sem_unlink("/lab8")` ao final.

Como neste laboratório as tarefas são threads do mesmo processo (e não processos distintos), foi utilizado o semáforo não nomeado (`sem_init/sem_destroy`), que é a forma recomendada nesse caso.

Para exclusão mútua, o valor inicial do semáforo deve ser 1 (e não 0, que deixaria todas as threads bloqueadas no primeiro `sem_wait`).

4.5 Compilação e execução

Os programas foram compilados e executados da seguinte forma:

```
gcc -Wall -o sem_semaforo sem_semaforo.c -pthread
gcc -Wall -o com_semaforo com_semaforo.c -pthread

./sem_semaforo          # 100 tarefas (padrao)
./sem_semaforo 1000    # numero de tarefas opcional na linha de
comando
./com_semaforo
```

5. Observações e Conclusões

Dificuldades

As principais dificuldades encontradas neste laboratório foram:

- garantir que as threads realmente disputassem o vetor ao mesmo tempo, o que exigiu liberá-las juntas por meio de uma variável de condição;
- identificar corretamente a região crítica (a atualização de cada posição), de forma a eliminar o erro sem serializar o programa inteiro;
- entender a diferença entre semáforo nomeado e não nomeado e qual valor inicial usar para exclusão mútua;
- analisar o comportamento não determinístico da versão sem semáforo, em que os erros mudam de posição e de quantidade a cada execução.

Resultados observados

Os testes permitiram observar que:

- sem semáforo, o acesso concorrente de 100 threads ao vetor global provocou erros de atualização perdida em centenas de posições (197, 76 e 763 em três execuções seguidas);
- os erros aparecem em blocos de posições consecutivas e em pontos diferentes a cada execução, comportamento típico de condição de corrida;
- com o semáforo binário protegendo a região crítica, todas as execuções terminaram com 0 posições com erro, mesmo com 1000 tarefas;
- a verificação automática de igualdade com o valor esperado (60) permitiu detectar com precisão as posições afetadas.

Conclusão final

O laboratório permitiu observar, de forma prática, o problema da condição de corrida e a sua correção por exclusão mútua.

Sem proteção, a operação ler–modificar–escrever sobre o vetor compartilhado está sujeita a perda de atualizações, e o resultado varia de forma não determinística entre execuções, confirmando o comportamento já discutido no LAB7 para threads sem sincronização.

Com o semáforo binário protegendo a região crítica, as tarefas continuam executando concorrentemente, mas o resultado final passa a ser sempre correto (60 em todas as 10.000 posições), ao custo de uma execução mais lenta devido à sincronização.

Portanto, o experimento mostra que o semáforo é um mecanismo eficaz de exclusão mútua: ele não elimina a concorrência, mas garante que a região crítica seja executada de forma atômica, tornando o programa correto independentemente da ordem em que o escalonador intercala as threads.